10 ALGORITMUS RR, ZPĚTNOVAZEBNÍ PLÁNOVÁNÍ, PRIORITNÍ PLÁNOVÁNÍ

10.1 ALGORITMUS RR – ROUND ROBIN

10.1.1 PRINCIP A PREEMPCE

- Jedná se o čistě preemptivní druh plánování, ve kterém mají všechny procesy shodnou prioritu
- Každý proces dostává procesor přidělen na přesně stanovený časový úsek, který nazýváme časové kvantum (q)
- Doba trvání časového kvanta se reálně pohybuje v řádu desítek až stovek milisekund

10.1.2 MECHANISMUS STŘÍDÁNÍ

- Jakmile běžícímu procesu vyprší jeho přidělené časové kvantum(a nestihne svou práci dokončit, je mu procesor násilně odejmut
- Proces je následně zařazen na konec systémové fronty připravených procesů a procesor je předán nejstaršímu procesu z této fronty
- Je-li ve frontě připravených procesů:
    - získá každý proces $\frac{1}{n}$ času procesoru
    - platí, že žádný z nich nebude na procesor čekat déle než $(n-1)q$ časových jednotek

10.1.3 EFEKTIVITA A ÚSILÍ

- Výkonnost tohoto algoritmu je silně závislá na velikosti zvoleného kvanta q:
    - Pokud je kvantum příliš velké:
        - ztrácí se výhoda cyklického střídání
        - plánování se začíná chovat jako metoda FCFS
    - Pokud je kvantum příliš malé:
        - vede to k neustálému přepínání kontextu procesů
        - to vytváří neúnosnou režijní zátěž pro celý systém
- Z hlediska nastavení platí pravidlo, že přibližně 80 % procesů by mělo svou dávku procesoru stihnout dokončit dříve, než jim časové kvantum vyprší
- Algoritmus RR nabízí velmi dobrý čas odezvy

10.2 ZPĚTNOVAZEBNÍ PLÁNOVÁNÍ

10.2.1 PRINCIP

- Zpětnovazební plánování řeší reálný problém, kdy dopředu neznáme potřebný čas běhu jednotlivých procesů
- Přistupuje k problému tak, že penalizuje procesy, které již na procesoru běžely příliš dlouho

10.2.2 VÍCEÚROVŇOVÉ FRONTY

- Způsob penalizace je úzce spjat s implementací víceúrovňových front, kde každá obsluhuje jinou úroveň priority
- Nad každou frontou samostatně běží určitý algoritmus (např. RR s různými kvanty)
- Pokud proces přečerpá své časové kvantum, dojde k přerušení a procesu je snížena priorita – je přeřazen do fronty o úroveň níže
    - Fronty s nižší prioritou mohou mít nastavena delší časová kvanta
    - Fronta s úplně nejnižší prioritou bývá obsluhována prostou metodou FCFS

10.2.3 PROBLÉM STÁRNUTÍ A ZRÁNÍ

- Tento model nese velké riziko „stárnutí“ pro procesy, které spadly do fronty s nejnižší prioritou – mohly by tam uváznout a neustále čekat
- Aby se tomu předešlo, uplatňuje se metoda tzv. zrání (aging)
    - Protichůdně k penalizaci se v určitých časových intervalech všem čekajícím procesům priorita zvyšuje
    - To zapříčiní, že se po čase proces přemístí zpět do front s vyšší prioritou

10.3 PRIORITNÍ PLÁNOVÁNÍ (HSFS)

10.3.1 PRINCIP

- S každým procesem je spojeno prioritní číslo (typu integer), které jasně vyjadřuje jeho preferenci při výběru z fronty připravených procesů
- Procesor je vždy přidělen tomu procesu, který má aktuálně nejvyšší prioritu
- Samotná reprezentace priority se může v různých OS lišit:
    - v Linuxu platí, že čím nižší číslo, tím vyšší priorita
    - ve Windows je implementace přesně opačná

10.3.2 VARIANTY PLÁNOVÁNÍ

10.3.2.1 NEPREEMPTIVNÍ PLÁNOVÁNÍ

- Vybranému procesu je přidělen čas procesoru
- Proces si jej drží tak dlouho, dokud svou požadovanou výpočetní dávku kompletně nedokončí

10.3.2.2 PREEMPTIVNÍ PLÁNOVÁNÍ

- Procesor je přidělen procesu s nejvyšší prioritou ve frontě
- Jakmile se do fronty zařadí nově příchozí proces, který má prioritu ještě vyšší než právě běžící proces, je aktuálně běžícímu procesu procesor násilně odejmut a okamžitě jej získá onen nový proces

10.3.3 PROBLÉM STÁRNUTÍ

- Pokud je v systému velké množství procesů s vysokou prioritou, mohou procesy s nižší prioritou „stárnout“ – neustále čekat, aniž by se někdy vůbec k procesoru dostaly
- K řešení se, stejně jako u zpětnovazebního plánování, využívá metoda zrání, kdy se čekajícím procesům s postupem času jejich priorita navyšuje